Grok

Module 1: Lexical Analysis and Compiler Basics

Question 1 (3 marks each):

- **A-1**: Find the lexemes in the programming statement `sum = a * (b - 10);`. Define tokens and patterns for the statement.
- **B-1**: Construct a regular expression for the language consisting of all strings ending with `00` over Σ = {0, 1}.
- **D-1**: Describe the input buffering scheme in a lexical analyzer.
- **E-1**: Define lexeme, tokens, and patterns using the source language statement `while (a > b) { result = a + b; }` as an example.
- **F-1**: Distinguish between the front end and back end of a compiler.
- **July 2023-1**: What is the relevance of input buffering in lexical analysis?

Question 2 (3 marks each):

- **A-2**: Explain the importance of sentinels in input buffering used in lexical analysis.
- **B-2**: Define tokens, lexemes, and patterns with suitable examples for each.
- **D-2**: Find regular definitions for tokens in the generated strings of the grammar: `S ⟶ if E then S | if E then S else S | E`, `E ⟶ T relop T | T`, `T ⟶ id | num`.
- **E-2**: Describe bootstrapping in compiler design using necessary diagrams.
- **F-2**: What are the different types of errors detected by a compiler?
- **July 2023-2**: Draw a transition diagram to represent relational operators.

Question 11 (14 marks each):

- **A-11**:
  a) Explain the working of different phases of a compiler. Illustrate with a source language statement.
  b) Explain different compiler construction tools.
- **B-11**:
  a) Explain in detail the various phases of the compiler with a neat diagram. Illustrate the output of each phase for the input `sum := a + b * 30` (where `a` and `b` are float variables).
  b) Apply bootstrapping to develop a compiler for a new high-level language `N` on machine `P`.
- **D-11**:
  a) Draw a transition diagram for designing a lexical analyzer for tokens such as arithmetic operators, relational operators, identifiers, and unsigned numbers.
  b) Write code for the lexical analyzer for the above design.
- **E-11**:
  a) Explain the different phases of a compiler for the source language statement `c = sum - row * 10`. Show the input and output at each compiler phase (assume `c`, `sum`, and `row` are floating-point variables).
  b) Draw transition diagrams for (i) relational operators, (ii) identifiers.
- **F-11**:
  a) Explain the functions of different phases of a compiler with a neat diagram. Illustrate the output of each phase for the input `a = b + c - 10`.
  b) Define lexeme, token, and pattern. Identify the lexemes that make up the tokens in the program segment:

```c
void swap (int i, int j) {
    int t;
    t = i;
    i = j;
    j = t;
}
```

- **July 2023-11**:
  a) What are the various phases of a compiler? Explain each phase in detail using the input statement `position := initial + rate * 60`.
  b) Differentiate tokens, patterns, and lexemes with an example.

Question 12 (14 marks each):

- **A-12**:
  a) Explain the role of transition diagrams in the recognition of tokens.
  b) Explain bootstrapping with an example.
- **B-12**:
  a) Explain the role of transition diagrams in the recognition of tokens. Draw the transition diagram for the regular definition: `relop ⟶ < | <= | = | <> | >= | >`.
  b) List and explain any three tools that help a programmer in building a compiler efficiently.
- **D-12**:
  a) Explain the LEX program structure with a sample program.
  b) Explain the YACC program structure with a sample program.
- **E-12**:
  a) Write a note on input buffering with necessary diagrams. Specify the advantages of using a two-buffer system and sentinels in input buffering.
  b) Explain any four compiler construction tools.
- **F-12**:
  a) Explain any four different compiler construction tools.
  b) What is bootstrapping in compiler development? Explain with an example.
- **July 2023-12**:
  a) Write short notes on compiler construction tools.
  b) Explain in detail about buffer pairs and sentinels.